

Praktikum 9: Machine Learning – Naïve Bayes

Muhammad Shiddiq 1 - 0110222199 ¹

¹ Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: muhammadshiddiq785@gmail.com

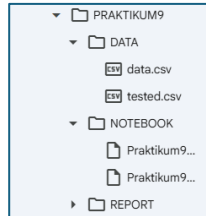
Abstract. Naive Bayes adalah keluarga algoritma klasifikasi probabilistik yang didasarkan pada Teorema Bayes dengan asumsi "naive" (naif) mengenai independensi antar fitur. Model ini bekerja berdasarkan prinsip bahwa keberadaan suatu fitur dalam kelas tertentu tidak terkait dengan keberadaan fitur lainnya. Dengan kata lain, model ini menghitung probabilitas suatu kelas ($P(C|X)$) berdasarkan probabilitas fitur individu ($P(X|C)$), dengan secara eksplisit mengasumsikan bahwa setiap fitur independen satu sama lain. Meskipun asumsi independensi fitur ini jarang terpenuhi sepenuhnya dalam skenario dunia nyata, model Naive Bayes seringkali menunjukkan kinerja yang sangat baik, terutama untuk tugas klasifikasi teks dan data berdimensi tinggi. Keunggulan utama model ini terletak pada kecepatan pelatihan dan prediksi yang luar biasa karena ia hanya perlu menghitung frekuensi probabilitas dari data, bukan melalui proses iteratif yang kompleks. Selain itu, model ini membutuhkan data pelatihan yang relatif lebih sedikit untuk estimasi parameter dan sangat mudah diimplementasikan, menjadikannya pilihan yang efisien dan baseline yang kuat dalam berbagai aplikasi pembelajaran mesin, seperti klasifikasi spam dan analisis sentimen.

1. Praktikum Mandiri – Mengklasifikasi seseorang mengidap kanker atau tidak

Tujuan utama klasifikasi pada dataset kanker payudara (data.csv) adalah untuk memprediksi secara akurat apakah tumor pasien bersifat Malignant (Ganas) atau Benign (Jinak), menggunakan Gaussian Naive Bayes sebagai model klasifikasi biner. Model ini didasarkan pada Teorema Bayes, secara "naif" mengasumsikan independensi antar fitur. Proses klasifikasi menggunakan (30) fitur numerik sebagai input—seperti rata-rata jari-jari, tekstur, dan kekompakan sel—yang memberikan gambaran morfologi tumor secara multidimensi. Algoritma bekerja dengan menghitung probabilitas diagnosis akhir berdasarkan probabilitas awal (prior) dan probabilitas kondisional (likelihood) dari setiap fitur. Keunggulan model ini adalah kecepatan pemrosesan dan efisiensi, serta kemampuannya untuk mencapai akurasi tinggi (berdasarkan hasil sebelumnya, sekitar (96.49\%)), menjadikannya alat yang potensial dan penting dalam meminimalkan kesalahan prediksi, terutama False Negative (salah mendiagnosis tumor Ganas sebagai Jinak), dalam konteks medis.

1.1 Membuat Folder

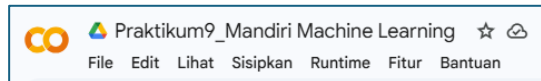
Langkah pertama kita harus membuat folder yang terstruktur dan juga rapih di google drive.



Gambar 1. Membuat folder di google drive, agar mudah untuk diakses.

1.2 Membuat file notebook google colab

Selanjutnya membuat file notebook di google colab untuk praktikum.



Gambar 2. Membuat file google colab

1.3 Menghubungkan google colab dengan google drive

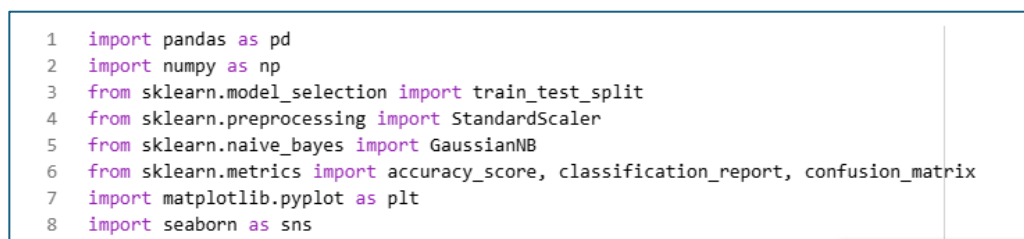
Selanjutnya menghubungkan google colab dengan google drive menggunakan perintah "From google.colab import drive
Drive.mount('/content/drive')".



Gambar 3. Menghubungkan google colab dengan google drive

1.4 Meng import library yang akan digunakan

Langkah pertama dalam praktikum ini adalah melakukan import terhadap seluruh library yang dibutuhkan.



Gambar 4. Mengimport library

1.5 Membaca dataset

Selanjutnya membaca dataset day.csv yang ada di google drive

```
1
2 df = pd.read_csv(path + "data.csv")
3 df
```

Gambar 5. Membaca dataset calonpembelimobil.csv.

Tabel 1. Berikut adalah hasil dataset yang telah dibaca.

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	con
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	
...	...	...	...	...	...	...	...	...
564	926424	M	21.56	22.39	142.00	1479.0	0.11100	
565	926682	M	20.13	28.25	131.20	1261.0	0.09780	
566	926954	M	16.60	28.08	108.30	858.1	0.08455	
567	927241	M	20.60	29.33	140.10	1265.0	0.11780	
568	92751	B	7.76	24.54	47.92	181.0	0.05263	

1.6 Mengecek informasi dataset

Selanjutnya mengecek informasi dataset yang dibaca, dari total, jumlah kolom, missing value, dan type data menggunakan perintah

“df.info()”

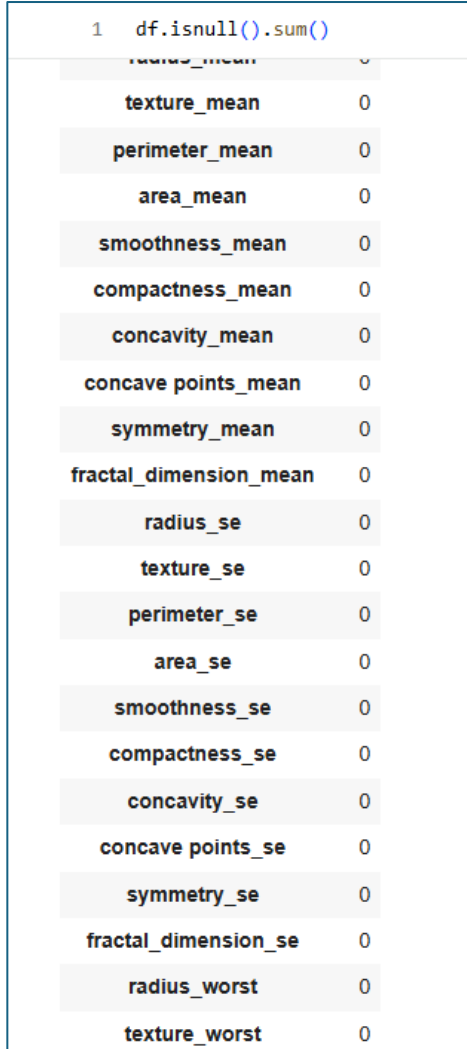
```
1 df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
#   Column                                Non-Null Count  Dtype  
---  -
0   id                                    569 non-null   int64  
1   diagnosis                            569 non-null   object  
2   radius_mean                          569 non-null   float64 
3   texture_mean                         569 non-null   float64 
4   perimeter_mean                       569 non-null   float64 
5   area_mean                            569 non-null   float64 
6   smoothness_mean                      569 non-null   float64 
7   compactness_mean                     569 non-null   float64 
8   concavity_mean                       569 non-null   float64 
9   concave points_mean                  569 non-null   float64 
10  symmetry_mean                        569 non-null   float64 
11  fractal_dimension_mean               569 non-null   float64 
12  radius_se                            569 non-null   float64 
13  texture_se                           569 non-null   float64 
14  perimeter_se                         569 non-null   float64 
15  area_se                              569 non-null   float64 
16  smoothness_se                        569 non-null   float64 
17  compactness_se                       569 non-null   float64 
18  concavity_se                         569 non-null   float64 
19  concave points_se                    569 non-null   float64 
20  symmetry_se                          569 non-null   float64 
21  fractal_dimension_se                 569 non-null   float64 
22  radius_worst                         569 non-null   float64 
23  texture_worst                        569 non-null   float64 
24  perimeter_worst                      569 non-null   float64 
25  area_worst                           569 non-null   float64 
26  smoothness_worst                     569 non-null   float64 
27  compactness_worst                    569 non-null   float64 
28  concavity_worst                      569 non-null   float64 
29  concave points_worst                  569 non-null   float64 
30  symmetry_worst                       569 non-null   float64 
31  fractal_dimension_worst              569 non-null   float64 
32  Unnamed: 32                          0 non-null     float64
```

Gambar 6. Mengecek informasi dataset.

1.7 Mengecek missing value

Selanjutnya mencari missing value pada data dengan perintah `df.isnull().sum()`

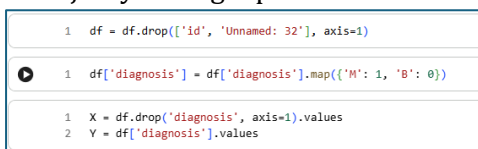


1	<code>df.isnull().sum()</code>	
	<code>radius_mean</code>	0
	<code>texture_mean</code>	0
	<code>perimeter_mean</code>	0
	<code>area_mean</code>	0
	<code>smoothness_mean</code>	0
	<code>compactness_mean</code>	0
	<code>concavity_mean</code>	0
	<code>concave points_mean</code>	0
	<code>symmetry_mean</code>	0
	<code>fractal_dimension_mean</code>	0
	<code>radius_se</code>	0
	<code>texture_se</code>	0
	<code>perimeter_se</code>	0
	<code>area_se</code>	0
	<code>smoothness_se</code>	0
	<code>compactness_se</code>	0
	<code>concavity_se</code>	0
	<code>concave points_se</code>	0
	<code>symmetry_se</code>	0
	<code>fractal_dimension_se</code>	0
	<code>radius_worst</code>	0
	<code>texture_worst</code>	0

Gambar 7. Mengecek missing value

1.8 Menghapus kolom yang tidak terpakai

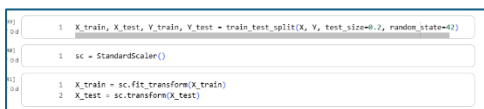
Selanjutnya menghapus kolom



1	<code>df = df.drop(['id', 'Unnamed: 32'], axis=1)</code>
1	<code>df['diagnosis'] = df['diagnosis'].map({'M': 1, 'B': 0})</code>
1	<code>X = df.drop('diagnosis', axis=1).values</code>
2	<code>Y = df['diagnosis'].values</code>

Gambar 8. Membersihkan missing value

1.9 Membagi data

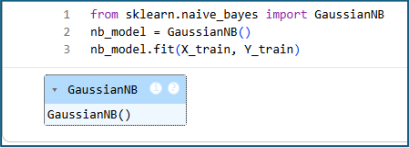


1	<code>X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)</code>
1	<code>sc = StandardScaler()</code>
1	<code>X_train = sc.fit_transform(X_train)</code>
2	<code>X_test = sc.transform(X_test)</code>

Gambar 9. Membagi data testing dan training

1.10 Membuat model Naive bayes

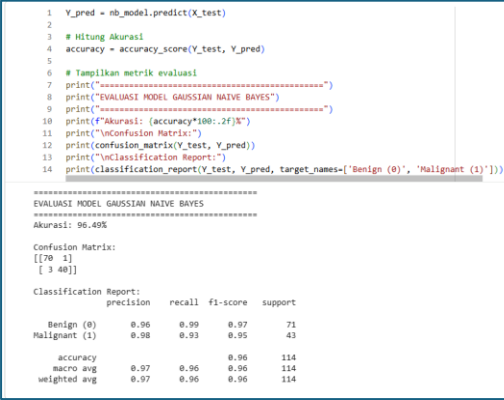
```
1 from sklearn.naive_bayes import GaussianNB
2 nb_model = GaussianNB()
3 nb_model.fit(X_train, Y_train)
```



Gambar 10. Membuat model

1.11 Menghitung akurasi

```
1 Y_pred = nb_model.predict(X_test)
2
3 # Hitung Akurasi
4 accuracy = accuracy_score(Y_test, Y_pred)
5
6 # Tampilkan metrik evaluasi
7 print("=====")
8 print("EVALUASI MODEL GAUSSIAN NAVE BAYES")
9 print("=====")
10 print(f"Akurasi: {accuracy*100:.2f}%")
11 print("\nConfusion Matrix:")
12 print(confusion_matrix(Y_test, Y_pred))
13 print("\nClassification Report:")
14 print(classification_report(Y_test, Y_pred, target_names= ['Benign (0)', 'Malignant (1)']))
```



```
=====
EVALUASI MODEL GAUSSIAN NAVE BAYES
=====
Akurasi: 96.49%

Confusion Matrix:
[[70  1]
 [ 3 40]]

Classification Report:
              precision    recall  f1-score   support

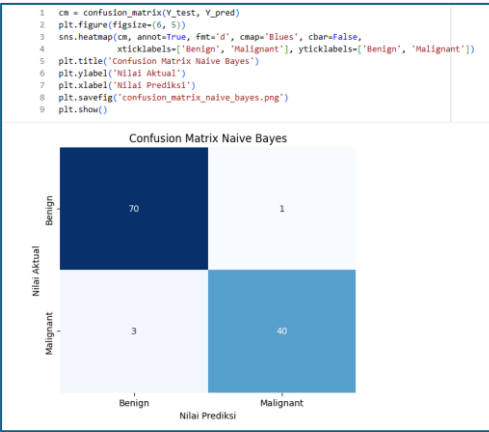
 Benign (0)       0.96       0.99       0.97         71
 Malignant (1)    0.98       0.93       0.95         43

   accuracy       0.97       0.96       0.96        114
  macro avg       0.97       0.96       0.96        114
 weighted avg       0.97       0.96       0.96        114
```

Gambar 11. Menghitung akurasi model

1.12 Membuat visualisasi data

```
1 cm = confusion_matrix(Y_test, Y_pred)
2 plt.figure(figsize=(6, 5))
3 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
4             xticklabels=['Benign', 'Malignant'], yticklabels=['Benign', 'Malignant'])
5 plt.title('Confusion Matrix Naive Bayes')
6 plt.ylabel('Nilai Aktual')
7 plt.xlabel('Nilai Prediksi')
8 plt.savefig('confusion_matrix_naive_bayes.png')
9 plt.show()
```



Confusion Matrix Naive Bayes

	Benign	Malignant
Benign	70	1
Malignant	3	40

Gambar 12. Membuat confusion matrix

1.13 Cross validation

```
1 # Lakukan 5-fold cross-validation
2 from sklearn.model_selection import cross_val_score
3 cv_nb = cross_val_score(nb_model, X, Y, cv=5, scoring='accuracy')
4
5 print("=====")
6 print("VALIDASI SILANG (5-FOLD)")
7 print("=====")
8 print("Skor Akurasi tiap Fold:", cv_nb)
9 print(f"Rata-rata Akurasi: {cv_nb.mean()*100:.2f}%")
10 print(f"Standar Deviasi: {cv_nb.std():.4f}")
11 print("=====")
=====
VALIDASI SILANG (5-FOLD)
=====
Skor Akurasi tiap Fold: [0.92185263 0.92185263 0.94736842 0.94736842 0.95575221]
Rata-rata Akurasi: 93.85%
Standar Deviasi: 0.0146
=====
```

Gambar 13. Membuat cross validation

Link Github : <https://github.com/Shid2iq/Machine-Learning>

Link Dataset : <https://www.kaggle.com/code/nisasoylu/naive-bayes-implementation-on-cancer-dataset>

Referensi:

- Munir, S., Seminar, K. B., Sudradjat, Sukoco, H., & Buono, A. (2022). The Use of Random Forest Regression for Estimating Leaf Nitrogen Content of Oil Palm Based on Sentinel 1-A Imagery. *Information*, 14(1), 10. <https://doi.org/10.3390/info14010010>
- Seminar, K. B., Imantho, H., Sudradjat, Yahya, S., Munir, S., Kaliana, I., Mei Haryadi, F., Noor Baroroh, A., Supriyanto, Handoyo, G. C., Kurnia Wijayanto, A., Ijang Wahyudin, C., Liyantono, Budiman, R., Bakir Pasaman, A., Rusiawan, D., & Sulastri. (2024). PreciPalm: An Intelligent System for Calculating Macronutrient Status and Fertilizer Recommendations for Oil Palm on Mineral Soils Based on a Precision Agriculture Approach. *Scientific World Journal*, 2024(1). <https://doi.org/10.1155/2024/1788726>